

STRINGS

También conocidos como **cadena**s, son un tipo de **datos compuestos por secuencia de caracteres individuales** (recordar que las cadenas son de tipo "str"). Estos datos son de secuencia inmutable, **no se pueden modificar** una vez ejecutado el programa.

Estos caracteres se imprimen como:

```
print ( " xyz " )
```

Si uno quiere agregar comillas dentro de la cadena puede que python no lo lea correctamente, para eso se usa el slash invertido (\ "x").

Importante: Los espacios, saltos de línea, signos de puntuación cuentan como un carácter más.

Uno puede crear una lista con caracteres strings con el método lista("x", "y", "z")

Indexación de caracteres:

Este índice nos permite seleccionar su carácter asociado haciendo referencia a él entre corchetes ([]) en el nombre de la variable que almacena la cadena.

Este método es útil si por ejemplo queremos acceder a caracteres en las últimas posiciones de una cadena con muchos caracteres de la cual no conocemos su longitud.

Caracteres :	P	y	t	h	o	n
Índice :	0	1	2	3	4	5
Índice inverso :	-6	-5	-4	-3	-2	-1

Métodos comunes

❖ Delimitación de strings:

Estos datos se delimitan mediante el uso de comillas simples (") o comillas dobles (").

❖ Variables y strings:

En cuanto a las variables, también pueden contener datos tipos "str"

Ejemplo:

```
x = "Este texto de ejemplo es muy largo,  
resultando en un código  
significativamente más largo."  
print ( x )
```

❖ Varias líneas en "str":

Para hacer esto basta con poner 3 comillas dobles o simples, así generamos saltos de línea, obviamente estos saltos contarán como carácter en la cadena.

Ejemplo:

```
x = """Este texto de ejemplo es muy largo,  
resultando en un código  
significativamente más largo.  
y finalmente termina con un punto y  
aparte."""  
print ( x )
```

❖ Imprimir un carácter de la cadena:

Para hacer esto se utiliza corchetes y el número del carácter de la cadena.

Ejemplo:

```
x = "Este jugo sabe rico"  
print ( x [ 3 ] )
```

out: t

Nota: Puedes también imprimir un intervalo de caracteres como:

```
print ( x [ 3:8 ] )
```

out: te jug

❖ Operaciones con strings:

- **Adicción:** La adicción nos ayuda con los strings a concatenar, si bien podremos hacerlo con una “ , ”, también podemos hacerlo con el (+).

Ejemplo:

```
m1 = "Hola" +  
"mundo"  
print ( m1 )  
out: Holamundo
```

Otra forma...

```
m1 = "Hola"  
m1 += "  
m1 += "mundo"
```

- **Multipliación:** Con esta operación podemos hacer repeticiones de un carácter sin la necesidad de escribirlo varias veces. El símbolo para hacerlo es (*)

Ejemplo:

```
m2 = "Bonito" * 4  
Print ( m2 )  
out: Bonito  
Bonito Bonito  
Bonito
```

- **Igual:** Esta operación sirve para comparar 2 cadenas de caracteres, tomar en cuenta que al comprar deben ser iguales en todo detalle ya sea mayúsculas y minúsculas.
El resultado de esta operación será un booleano, si no se parecen dará False y si se parecen será True.

❖ Minúsculas y mayúsculas:

Para convertir una cadena en totalmente en minúsculas se utiliza la función (.lower()) y para mayúsculas se usa (.upper()).

Ejemplo:

```
m7 = "HOLA MUNDO"
m7 = m7.lower()
print(m7)
```

out: hola mundo

❖ Reemplazar:

Para reemplazar un carácter de la cadena por otro o por una cadena se usa (.replace())

Ejemplo:

```
m8 = "HOLA MUNDO"
m8 = m8.replace("L", "pizza")
print(m8)
```

out: HOpizzaA MUNDO

❖ Encontrar:

Para buscar el número de índice de un carácter en una cadena se utiliza el método (.find())

Ejemplo:

```
m5 = "Hola Mundo"
m5 = m5.find("Mundo")
print(mensaje5a)
```

out: 5

❖ Contar caracteres en una cadena:

Para ver la 'longitud' o cantidad de caracteres de una cadena se utiliza el método (len(x))

Ejemplo:

```
text = "Este texto de ejemplo es muy largo, resultando en un código significativamente más largo."
print(len(text))
```

out: 89

❖ Preguntar por una subcadena de una cadena de caracteres:

Este método sirve para preguntar por una letra en la cadena, ara esto se usa (in)

Ejemplos:

```
text = " Este texto de ejemplo es muy largo, resultando en un código significativamente más largo."
print("largo" in text)
```

out: True

otra forma:

```
text = "Este texto de ejemplo es muy largo y resulta en un código significativamente más largo."
if "largo" in text:
    print (" Sí, 'largo' aparece en este extracto de código.")
```

out: True

❖ Para descartar la existencia de una subcadena:

text = " Este texto de ejemplo es muy largo y resulta en un código significativamente más largo."

```
print("corto" not in text)
```

out: False

❖ Para dividir strings:

Esta function sirve para separar en subcadenas la cadena principal, el método es (.split()). Tomar en cuenta que el split toma como referencia un parámetro que puede ser cualquier carácter presente en la cadena, el mas usado es el espacio, pero también pueden ser otros.

Ejemplo:

```
text = "Esto es un ejemplo"
```

```
print(text.split())
```

out: ["Esto", "es", "un", "ejemplo"]

❖ Emitir bucle con string:

Para hacer bucles se usa lo siguiente:

Ejemplo:

```
for x in "diente de león":  
    print ( x )  
for caracter in "diente de león":  
    print(caracter)
```

❖ Método máx. y min:

Estos métodos utilizan el índice **Unicode** para caracteres strings. Cuando se utilice va a analizar el índice Unicode de cada carácter. **Ejemplo:**

```
cadena= "bbtrickz"  
print(min(cadena))  
print(max(cadena))
```

out: max(b), min(z)

❖ Ordenar una cadena alfabéticamente:

Se usa el método (sorted())

Ejemplo:

```
cadena = "asdlk"  
print(sorted(cadena))
```

out: ['a', 'd', 'k', 'l', 's']